

# Continuous Delivery

Portfolio evidence: how I applied CD principles to LifeForest

## Learning outcome claim

I applied Continuous Delivery in LifeForest by keeping the project in a releasable state through automated build, test, lint, type-check, dependency-audit, artifact, Docker, and deployment-preparation steps. Production deployment is still manual, so this is CD rather than full Continuous Deployment.

## What Continuous Delivery Means

Continuous Delivery means code changes are automatically validated and prepared for release. The final production release remains a manual decision, but the project stays close to deployable.

## CD Pipeline Evidence

| CD practice                   | LifeForest implementation                                | Evidence location                          | Release value                          |
|-------------------------------|----------------------------------------------------------|--------------------------------------------|----------------------------------------|
| <b>Automated trigger</b>      | Runs on push to main and pull requests targeting main.   | .github/workflows/ci.yml                   | Important changes are validated.       |
| <b>Backend build</b>          | Sets up Java 21, runs Gradle assemble, uploads JAR.      | .github/workflows/ci.yml                   | Backend becomes a releasable artifact. |
| <b>Backend tests</b>          | Runs ./gradlew test after build succeeds.                | backend/src/test and CI                    | Business/API behaviour is checked.     |
| <b>Dependency security</b>    | Runs OWASP Dependency-Check and uploads reports.         | backend/build.gradle and CI                | Risky dependencies are visible.        |
| <b>Frontend validation</b>    | Runs npm ci, lint, typecheck, optional tests, npm audit. | frontend/package.json and CI               | Frontend quality is checked.           |
| <b>Docker packaging</b>       | Dockerfiles define repeatable environments.              | backend/Dockerfile and frontend/Dockerfile | Builds are portable.                   |
| <b>Manual release control</b> | Docker Hub push and VM deployment are documented.        | README.md                                  | Release timing stays controlled.       |

## CD, Not Continuous Deployment

| Aspect                    | What LifeForest does                                                             | What this shows                                   |
|---------------------------|----------------------------------------------------------------------------------|---------------------------------------------------|
| <b>Automatic checks</b>   | Build, tests, linting, type checks, dependency checks, and audits are automated. | The project is prepared for release continuously. |
| <b>Production release</b> | Docker image push and VM deployment are documented but manually triggered.       | The team controls when the release goes live.     |
| <b>Risk management</b>    | CI fails on important checks while deployment stays deliberate.                  | This matches Continuous Delivery.                 |

## Docker and Release Preparation

- Backend Dockerfile builds the Spring Boot JAR with Java 21 and runs it in a smaller runtime image.
- Frontend Dockerfile installs Node dependencies and starts Expo on predictable ports.
- compose.yaml starts PostgreSQL and backend with environment-based configuration.

- README documents Docker startup, Docker Hub image pushing, and VM deployment.

## Reflection and Next Improvements

The strongest CD evidence is the GitHub Actions workflow combined with Dockerfiles, Docker Compose, and documented VM deployment commands. The main improvement would be automatically building and publishing Docker images after successful CI, then adding a staging environment before manual production release.

- Add a GitHub Actions job that builds Docker images after tests pass.
- Push versioned images to Docker Hub or GitHub Container Registry after approval.
- Add a staging environment and smoke tests against the same image intended for production.